CSE3031: Computer Architecture
Lab 2: Cache Simulator

Benjamin Mannal
New Mexico Institute of Mining and Technology

March 24, 2026

1 Introduction

In this lab we intended to learn more about how the cache works in computer architecture with its multiple cache policies. We intended to build a working cache simulator that can use multiple designs while intaking a configuration and trace file. To do this we implemented different cache designs such as direct mapped vs fully associative, random replacement vs FIFO replacement, and the different write policies such as write through/write no allocate and write back/write allocate. Through implementing these in C we plan to explore how different cache designs will impact a variety of statistics.

2 Method

2.1 Implementation

The cache simulator for this lab was implemented in C. The program takes two command-line arguments: a configuration file and a trace file, and outputs statistics to a file with the `.out` extension.

2.1.1 Data Structures

The cache simulator is organized around several core data structures that model the cache hierarchy and track performance. A CacheLine represents an individual cache line and stores a valid bit, the address tag, and a FIFO order counter used for replacement decisions. A CacheSet represents one set within a set-associative cache and contains an array of CacheLine structures together with a FIFO counter for managing replacement order across the set. The main Cache structure contains an array of CacheSet structures along with the cache's configuration parameters, including line size, associativity, number of sets, replacement policy, miss penalty, and write-allocate policy. In addition, a Statistics structure records performance data such as total accesses, hits, misses, separate load and store counts, total execution cycles, and memory access latency.

2.2 Usage

To use the cache simulator:

1. Compile: `make`
2. Execute: `./cachesim <config_file> <trace_file>`
3. Output appears in `<trace_file>.out`

The output file contains five lines:

1. Total hit rate (%)
2. Load hit rate (%)

3. Store hit rate (%)
4. Total runtime (cycles)
5. Average memory access latency (cycles)

3 Results

For the tables below, each cache configuration and trace was tested individually and then input into the table with the corresponding values from the .out file.

3.1 2way-nwa.conf

Trace File	Total Hit Rate	Load Hit Rate	Store Hit Rate	Total Run Time (Cycles)	Avg Mem Latency
gcc.trace	93.83%	98.76%	85.98%	3766584	5.32 cycles
gzip.trace	66.71%	50.24%	99.58%	12292034	24.4 cycles
mcf.trace	1.05%	95.61%	0.27%	51383748	70.26 cycles
swim.trace	92.61%	99.07%	75.3%	2745182	6.17 cycles
twolf.trace	98.86%	99.8%	96.47%	1836237	1.80 cycles

Table 1: 2-way set-associative, no-write-allocate configuration results

3.2 2way-wa.conf

Trace File	Total Hit Rate	Load Hit Rate	Store Hit Rate	Total Run Time (Cycles)	Avg Mem Latency
gcc.trace	98.76%	99.44%	97.67%	1985994	1.86 cycles
gzip.trace	66.82%	50.26%	99.88%	12254304	24.22 cycles
mcf.trace	75.24%	96.15%	75.06%	13619238	18.33 cycles
swim.trace	97.85%	99.69%	92.92%	1631622	2.50 cycles
twolf.trace	99.65%	99.85%	99.09%	1567297	1.24 cycles

Table 2: 2-way set-associative, write-allocate configuration results

3.3 4way-fifo.conf

Trace File	Total Hit Rate	Load Hit Rate	Store Hit Rate	Total Run Time (Cycles)	Avg Mem Latency
gcc.trace	98.72%	99.38%	97.66%	1869014	1.63 cycles
gzip.trace	66.82%	50.25%	99.88%	9062604	17.58 cycles
mcf.trace	75.23%	96.16%	75.06%	10017608	13.38 cycles
swim.trace	87.82%	99.66%	92.91%	1506002	2.08 cycles
twolf.trace	99.65%	99.86%	99.09%	1534377	1.17 cycles

Table 3: 4-way set-associative, FIFO replacement configuration results

3.4 4way-rand.conf

Trace File	Total Hit Rate	Load Hit Rate	Store Hit Rate	Total Run Time (Cycles)	Avg Mem Latency
gcc.trace	98.70%	99.36%	97.63%	1875314	1.65 cycles
gzip.trace	66.82%	50.25%	99.88%	9062604	17.58 cycles
mcf.trace	75.23%	96.16%	75.06%	10017708	13.38 cycles
swim.trace	97.78%	99.61%	92.90%	1511602	2.11 cycles
twolf.trace	99.65%	99.86%	99.10%	1535027	1.17 cycles

Table 4: 4-way set-associative, random replacement configuration results

3.5 large-dm.conf

Trace File	Total Hit Rate	Load Hit Rate	Store Hit Rate	Total Run Time (Cycles)	Avg Mem Latency
gcc.trace	95.83%	98.72%	91.18%	2614164	3.08 cycles
gzip.trace	66.70%	50.23%	99.57%	9091004	17.64 cycles
mcf.trace	1.03%	93.23%	0.27%	36997808	50.48 cycles
swim.trace	93.43%	99.36%	77.56%	2171902	4.28 cycles
twolf.trace	98.84%	99.68%	96.59%	1730027	1.57 cycles

Table 5: Large direct-mapped cache configuration results

3.6 mega.conf

Trace File	Total Hit Rate	Load Hit Rate	Store Hit Rate	Total Run Time (Cycles)	Avg Mem Latency
gcc.trace	99.34%	99.68%	98.8%	1877364	1.64 cycles
gzip.trace	66.84%	50.26%	99.93%	17031854	34.05 cycles
mcf.trace	87.61%	97.58%	87.53%	10021608	13.38 cycles
swim.trace	98.86%	98.82%	96.29%	1521502	2.14 cycles
twolf.trace	99.81%	99.91%	99.53%	1546327	1.19 cycles

Table 6: Mega cache configuration results

3.7 small-dm.conf

Trace File	Total Hit Rate	Load Hit Rate	Store Hit Rate	Total Run Time (Cycles)	Avg Mem Latency
gcc.trace	93.85%	96.40%	89.74%	3125464	4.07 cycles
gzip.trace	66.69%	50.22%	99.57%	9093354	17.65 cycles
mcf.trace	1.02%	91.64%	0.27%	37004258	50.48 cycles
swim.trace	91.61%	97.36%	76.24%	2447452	5.19 cycles
twolf.trace	98.26%	99.05%	96.16%	1869027	1.86 cycles

Table 7: Small direct-mapped cache configuration results

3.8 medium-dm.conf

Trace File	Total Hit Rate	Load Hit Rate	Store Hit Rate	Total Run Time (Cycles)	Avg Mem Latency
gcc.trace	95.36%	98.17%	90.84%	2736614	3.32 cycles
gzip.trace	66.70%	50.23%	99.58%	9091704	17.65 cycles
mcf.trace	1.03%	93.18%	0.27%	36998158	50.48 cycles
swim.trace	92.92%	92.93%	77.14%	2248952	4.54 cycles
twolf.trace	98.73%	99.55%	96.53%	1759177	1.64 cycles

Table 8: Medium direct-mapped cache configuration results

3.9 Performance Graphs

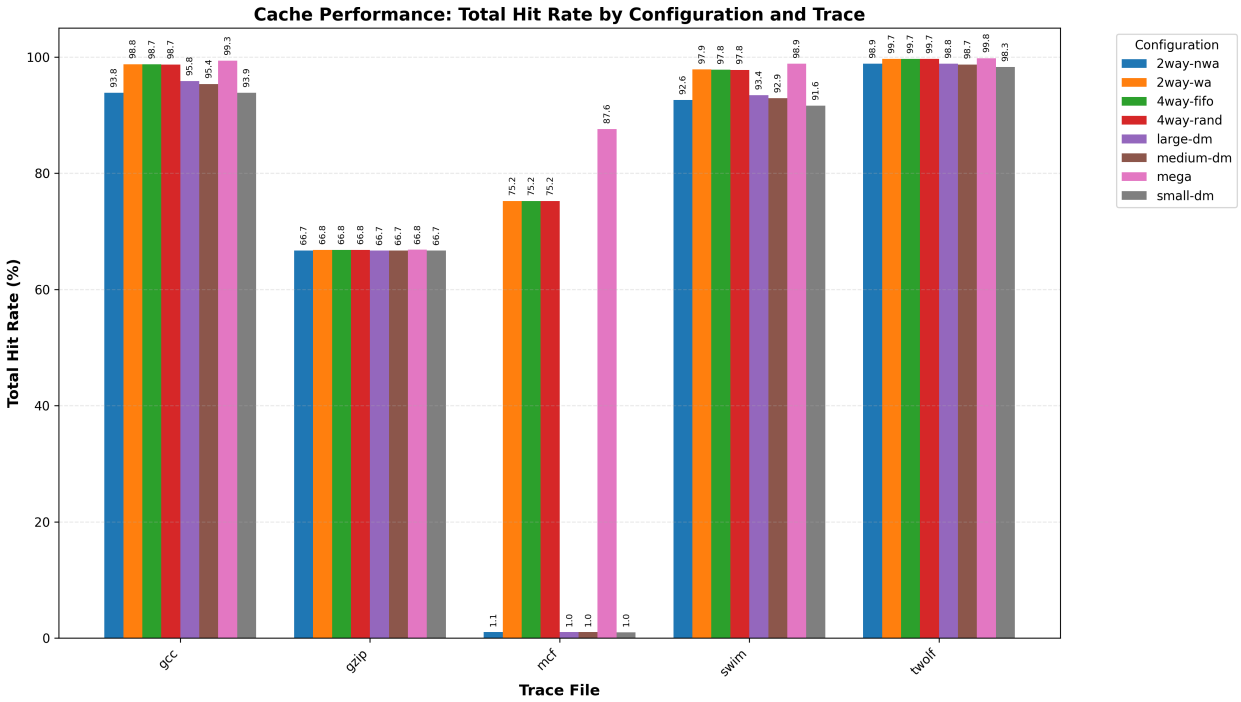

Figure 1: Total Hit Rate for Each Chosen Cache Config

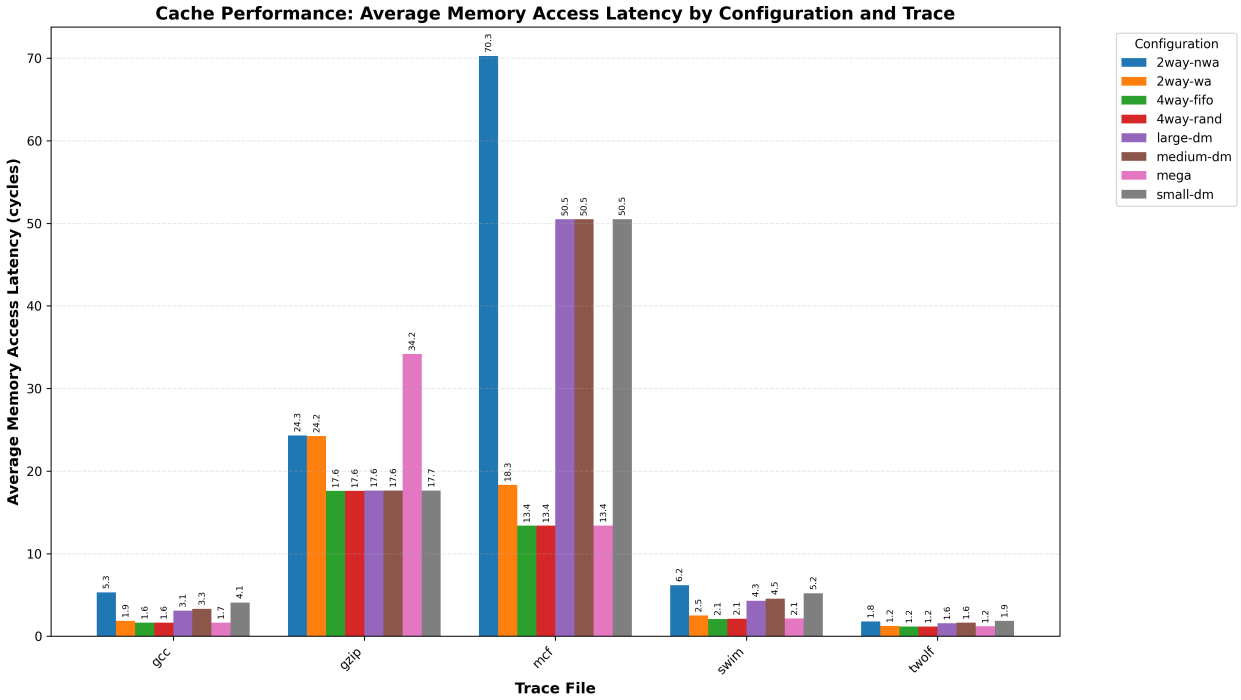

Figure 2: Average Memory Access Latency for Each Chosen Cache Config

3.10 Analysis

The results demonstrate several important trends in cache performance:

Impact of Write Policy: Comparing 2way-nwa and 2way-wa configurations reveals that write-allocate policies significantly improve performance on most traces. For example, on the gcc trace, write-allocate achieves 98.76% hit rate versus 93.83% for no-write-allocate, resulting in nearly half the execution time (1,985,994 vs 3,766,584 cycles).

Associativity Effects: Higher associativity generally reduces conflict misses. The 4-way configurations show improved or similar performance compared to 2-way configurations across most traces. The mcf trace particularly benefits from higher associativity, jumping from 1.05% (2way-nwa) to 75.23% (4way-fifo/rand).

Replacement Policy Comparison: FIFO and random replacement policies show nearly identical performance across all tests, suggesting that for these workloads, the replacement policy has minimal impact compared to cache size and associativity.

Cache Size Impact: Larger caches generally achieve higher hit rates. The mega configuration achieves the best overall hit rates, including 99.34% on gcc and 87.61% on mcf. However, gzip shows an interesting anomaly where the mega cache has higher latency due to its larger miss penalty.

Workload Characteristics: Different traces exhibit vastly different cache behavior:

- **gcc, swim, twolf**: High spatial/temporal locality, achieving ¿90% hit rates on most configurations
- **gzip**: Consistently 67% hit rate across all configurations, suggesting limited cache-friendly access patterns
- **mcf**: Extremely poor performance on direct-mapped caches (¡2% hit rate), but dramatically improves with associativity (75-87%)

4 Discussion

The experimental results highlight key trade-offs in cache design. While larger, more associative caches generally perform better, the mcf trace demonstrates that some workloads are particularly sensitive to conflict misses and require set-associative organization to achieve acceptable performance.

The write policy choice proved critical for overall performance. Write-allocate configurations consistently outperformed no-write-allocate versions, as bringing written data into the cache on misses provides opportunities for temporal locality on subsequent accesses.

Interestingly, the choice between FIFO and random replacement had negligible impact on these benchmarks. This suggests that for typical workloads, sophisticated replacement algorithms may provide diminishing returns compared to simply increasing cache size or associativity.

This lab successfully implemented a configurable cache simulator capable of modeling various cache organizations and policies. The experimental results confirmed theoretical predictions about cache behavior: larger caches and higher associativity generally improve

hit rates, write-allocate policies enhance performance, and different workloads exhibit vastly different cache sensitivity.

The most significant finding was the dramatic performance variance across workloads, with some traces (like mcf) showing over 70 percentage point improvements in hit rate when moving from direct-mapped to set-associative caches, while others (like gzip) remained largely insensitive to cache configuration.

These insights demonstrate the importance of understanding workload characteristics when designing cache hierarchies and highlight the ongoing challenges in creating cache systems that perform well across diverse application domains.